Content Analysis & Metadata

Multimedia Retrieval · Chapter 9

Multimedia Retrieval - 2026 - Uni Basel

Roger Weber

Contents

9 - Content Analysis & Metadata 1
The Semantic Gap 2
From Text to Media 2
The Gap Across Modalities 2
Why It Matters 3
Ways to Bridge the Gap 4
Metadata and Its Limits 5
The Keyword Path 5
Metadata Standards 6
Curated Catalogues 6
Manual Annotation Does Not Scale 7
Harvesting Context from the Web 9
Can We Trust Metadata? 10
Automated Feature Extraction 12
The Abstraction Ladder 12
Perceptual Features 13
Classification: From Features to Labels 14
Neural Networks: Key Steps 14
Multimodal Transformers 17
Describe, Then Search Text 17
The Eighth Obstacle: Hallucination 18
A Shared Embedding Space 18
Choosing a Path 20
Time-Based Media and Specialized Solutions 21
Time as a New Dimension 21
Specialized Solutions 22
Summary 24
Approach Comparison 24
Key Takeaways 24
Key Formulas 25
Self-Check Questions 25
Further Reading 26

9 - Content Analysis & Metadata

The previous chapters built a complete toolkit for text. An inverted file finds every document that contains a query term, BM25 ranks them, embeddings match “heart disease” with “cardiology”, vector indexes search millions of embeddings in milliseconds, and a language model turns the top passages into an answer. Now point the same system at a photo archive, a music catalogue, or a video library. The query is still text, but the documents are pixels, audio samples, and video frames. Nothing in a JPEG file says “Taj Mahal”. Can we do for images, audio, and video what we did for text?

The answer comes in steps. The obvious fix is to describe every item with keywords and reuse text search. It works, and curated catalogues such as IMDb live on it, but human annotation does not scale and metadata from the wild cannot be trusted. The lasting answer is to extract descriptions from the content itself: perceptual features that describe what humans see and hear, classifiers that turn those features into labels, and multimodal transformers that either write a description for the text index or place images and text in one shared vector space. Audio and video add time as a new dimension, and some tasks, such as naming a song from a noisy phone recording, need solutions of their own. The chapters that follow open the toolbox for each modality; this chapter draws the map.

What you’ll learn

- Explain the semantic gap and why it is wider for images, audio, and video than for text.
- Describe the types and facets of metadata and standards such as EXIF, IPTC, Dublin Core, and ID3, and explain why manual annotation neither scales nor can always be trusted.
- Distinguish the levels of automated content analysis, from perceptual features through learned classifiers to multimodal transformers.
- Compare the two transformer paths: generating text for a keyword index versus embedding text and media in a shared vector space such as CLIP.
- Explain why time-based media need segmentation into retrieval units, and recognize when a task calls for a specialized solution such as audio fingerprinting.

Prerequisites

- Text retrieval with keywords: Classical Text Retrieval and Index for Text Retrieval, including facets and filters in Filtering, Facets, and Compression.
- Embeddings and nearest-neighbor search: Building a Semantic Search System and Vector Search.
- Chunking long documents into retrieval units: Chunking Strategies.
- Evaluating classifiers and choosing thresholds: Evaluating Text Classifiers and Scores, Thresholds, and ROC Curves.

The Semantic Gap

Consider a photo of the Taj Mahal in a large image collection. The Taj Mahal, located in Agra, India, is a mausoleum that Emperor Shah Jahan built in memory of his wife Mumtaz Mahal, who died in 1631. It is one of the most recognized buildings in the world and a UNESCO World Heritage Site. Users who look for such a picture type very different queries: “building, outdoor, sky”, “mausoleum, dome, minaret”, “Taj Mahal”, “UNESCO World Heritage Site”, or even “where is Mumtaz Mahal buried?”. A text retrieval system answers each of these queries by matching words. The photo contains no words.

From Text to Media

Text carries its meaning on the surface. Words and sentences are symbols that people invented to express concepts, so a query and a document share the same symbol system. Even the vocabulary mismatch between “heart disease” and “cardiology treatment”, which defeated keyword search for decades, was largely closed by the embeddings of Semantic Similarity with Transformers: the model learned that both phrases mean similar things. In every case, the starting point was a sequence of words.

An image starts from a grid of light intensities. A photo of 1024 by 768 pixels with three color channels is a list of $1024 \cdot 768 \cdot 3 = 2,359,296$ numbers between 0 and 255. None of these numbers is “dome”, “sky”, or “Taj Mahal”. Figure 9.1 contrasts the two views: what the machine processes is a patch of color values, and what the user enters is a list of concepts at very different levels of abstraction.

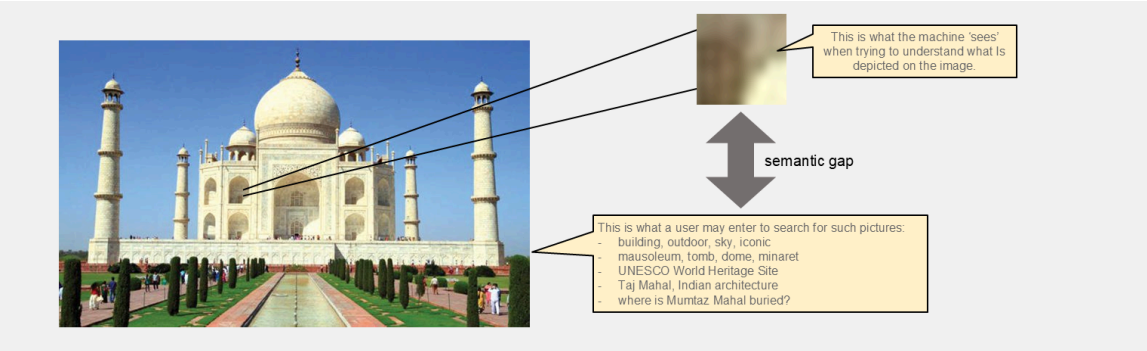


Figure 9.1: The semantic gap: the pixel values a machine processes versus the queries a user enters for the same photo.

The **semantic gap** is the distance between the low-level features that a system can compute from the signal and the high-level concepts that people use to describe and search for content. The term became standard with the survey by Smeulders et al. in 2000, listed in the Further Reading of the chapter summary, but the problem applies to every non-text medium.

The queries in Figure 9.1 reveal that the gap has two parts. Concepts such as “building”, “sky”, and “dome” are visible in the image: a system that interprets the pixels well enough can find them. Concepts such as “UNESCO World Heritage Site” or “where is Mumtaz Mahal buried?” are not in the pixels at all. They require knowledge about the world that the photo does not contain. A system can close the first part by analyzing the content; the second part needs context from outside the content. Such context comes from metadata, from the web page around the photo, or from the world knowledge that a large model absorbed during training, as Multimodal Transformers shows.

The Gap Across Modalities

The semantic gap appears in every medium, but its shape differs:

- **Text**: the gap is small because the symbols already carry meaning. Keyword search bridged it for decades, and embeddings closed most of the remaining vocabulary mismatch.
- **Images**: a blue region can be sky, water, or fabric. Interpreting the pixels requires recognizing objects, scenes, and people, and relating them to each other.
- **Audio**: a CD stores 44,100 samples per second and channel, so a three-minute stereo song is $44,100 \cdot 180 \cdot 2 = 15,876,000$ amplitude values. A user asks for “a sad piano piece” or “the song that is playing right now”.

- **Video**: frames, sound, and subtitles unfold together over time. At 25 frames per second, a one-hour film has 90,000 frames, and a user wants the one scene in which a character says a certain sentence.

For text, the meaning is in the data and needs to be matched. For the other media, the meaning must first be extracted from the data before anything can be matched. This extraction is called **content analysis**, and it is the subject of this chapter and the chapters that follow.

> **The semantic gap is not a vocabulary mismatch**
>
> It is tempting to equate the semantic gap with the vocabulary mismatch of text retrieval ("car" versus "automobile"). In text, query and document use the same kind of symbols, and the problem is that different words express the same concept. In images, audio, and video, the document contains no symbols at all. The system must first produce a description before it can face the vocabulary mismatch.

Why It Matters

Three application areas show why content analysis has become essential.

Social media and content platforms. Users upload billions of images and videos every day. Early platforms recommended content mostly through collaborative filtering: users who liked the same items in the past get similar suggestions. A new clip has no viewing history yet, so behavioral signals cannot place it. Behavior alone cannot tell whether a clip shows playful kittens or a lion in a documentary. Feed algorithms such as the one behind TikTok combine behavioral signals with content signals: captions, hashtags, the sound track, and visual features of the video itself. The feeds subsection of Learning to Rank describes how such feeds are ranked; this chapter is about where the content signals come from.

Media archives and newsrooms. Broadcasters and news agencies hold millions of photos, video clips, audio recordings, and articles collected over decades. Figure 9.2 shows the tape archive of the Danish Broadcasting Corporation. When breaking news arrives, editors have minutes to assemble the material. Suppose Roger Federer announces the end of his career. The evening news wants the highlights of 24 years on tour, from the first Wimbledon title to the last match. A sports broadcaster holds millions of hours of video, and thousands of them show Federer playing tennis. Finding clips with Federer is not enough: the editors need the few that are worth showing.

Traditionally, archivists answered this question at filing time. For each new clip, they judged its value for future use, described it with keywords such as the names of the people, the event, and the date, and filed it into tagged folders. The folders worked like an inverted file: the folder "Federer" pointed to the clips worth keeping, so an editor could pull the material within minutes. This metadata was of high quality, but production volumes outgrew the cataloguers long ago, and a decision made at filing time cannot anticipate every future question. Modern archives use face recognition to identify people across thousands of hours of footage, scene detection to cut videos into segments, speech-to-text to make interviews searchable, and multimodal embeddings so that a query for "earthquake" finds relevant clips even without a matching keyword.



The archive of the Danish Broadcasting Corporation
https://www.flickr.com/photos/kulturarvsprojektet/6498650083/

Figure 9.2: The tape archive of the Danish Broadcasting Corporation: when breaking news arrives, the right material must come out within minutes.

Music and audio services. Shazam names a song from a few seconds recorded in a noisy bar. Spotify recommends tracks that fit a mood, such as "music for a rainy afternoon", or that sound like a given song. Both must connect acoustic patterns with what listeners mean, and both return in Time-Based Media and Specialized Solutions.

Ways to Bridge the Gap

Several complementary strategies connect media content with the concepts of the user. They form the road map of this chapter. Figure 9.3 shows them side by side.

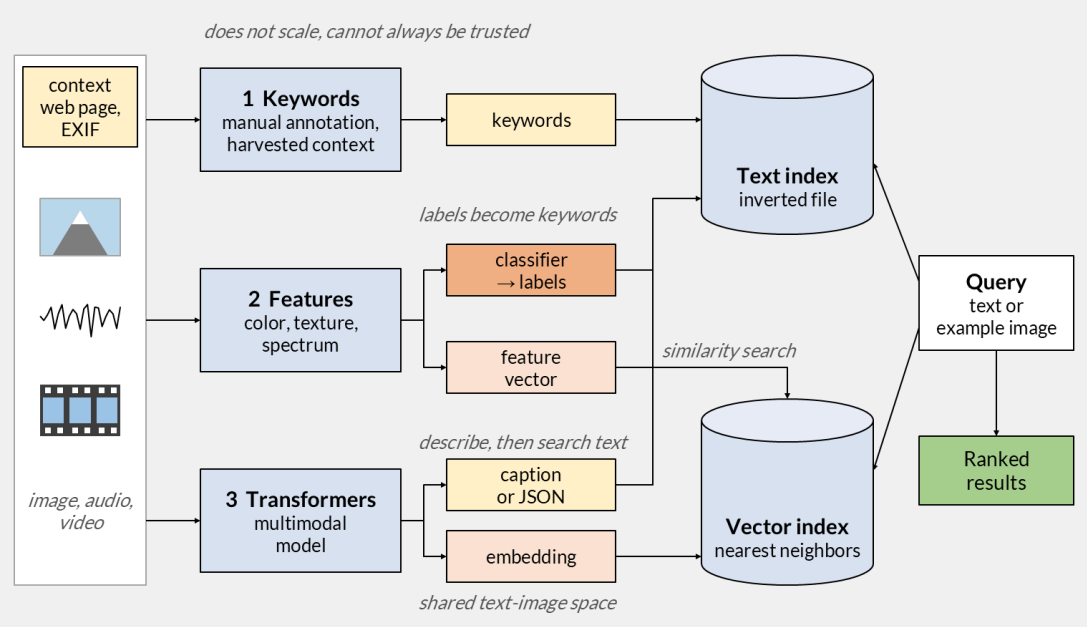


Figure 9.3: Three paths from media content to search: keywords, features, and transformers.

1. **Keywords (metadata).** Describe each item with text and search the text with the toolkit of the previous chapters. Humans write the descriptions, or the system harvests them from the context, such as the web page around an image. Metadata and Its Limits shows how far this path goes and where it breaks.
2. **Features.** Compute perceptual features that describe how the content looks or sounds, and compare items by feature similarity, for example "find images with similar colors". Classifiers turn features into labels such as "person" or "outdoor", which feed back into the keyword path. Automated Feature Extraction introduces this level.
3. **Transformers.** Multimodal transformers either generate a rich description of the content, which again feeds the keyword path, or map media and text into a shared embedding space, so that a text query can be compared directly with an image. Multimodal Transformers explains both options.

Audio and video add time, and some tasks need specialized solutions. Time-Based Media and Specialized Solutions covers both. The remaining chapters of this part then work out the perceptual features for images, audio, and video in detail.

Metadata and Its Limits

The quickest way across the semantic gap avoids the content altogether. If the Taj Mahal photo comes with the description “Taj Mahal, mausoleum in Agra, India, built by Shah Jahan”, then the query “Taj Mahal” matches the description and the photo is found. Every tool from the previous chapters applies unchanged: tokenization, an inverted file, BM25, embeddings, and filters. This section follows this keyword path from its strengths to its two fundamental problems: someone has to write the descriptions, and the descriptions are not always true.

The Keyword Path

Metadata is additional information attached to a media item that describes its content, its context, or its technical properties. For retrieval, the important property of metadata is that it is text, or at least structured values such as dates and coordinates. Once each item has metadata, a multimedia search becomes a text search over the metadata, and the semantic gap is bridged by whoever wrote it.

Metadata comes from three sources. Humans can annotate each item manually. The system can capture or harvest metadata automatically: the camera writes the date and the GPS position into the file, and a web page provides captions and surrounding text. Finally, machine learning models can analyze the content and produce labels or descriptions. This section covers the first two sources; the third is the subject of Automated Feature Extraction and Multimodal Transformers.

Metadata values describe the content at three levels:

- **Generic** metadata names general classes: “building”, “sky”, “outside”, “summer”.
- **Specific** metadata names individual entities and facts: “Taj Mahal”, “Agra”, “2006”, “UNESCO World Heritage Site”.
- **Abstract** metadata names subjective concepts and interpretations: “love”, “devotion”, “iconic”, “beautiful”.

Annotation schemas organize these values into **facets**, each describing one aspect of the item. Figure 9.4 annotates the Taj Mahal photo along five facets: objects (from “building, water, sky” to “Taj Mahal”), space (from “outside” to “India, Uttar Pradesh, Agra”), events (from “tourism” to a specific expert meeting in 2006), time (“summer, daytime” and “2006”), and context (topic, related persons, abstract concepts such as “love, death, devotion”, and the history of the building). Each facet holds generic and specific values side by side.



Object Facet	Value
Generic Object Instance	building, water, sky
Generic Object Class	mausoleum, tomb, dome, minaret
Specific Named Object Class	UNESCO World Heritage Site (since 1983)
Specific Named Object Instance	Taj Mahal

Spatial Facet	Value
Generic Location	outside
Specific Location Hierarchy	India, Uttar Pradesh, Agra

Event / Activity Facet	Value
Generic Event/Activity	tourism, attraction
Specific Event Instance	International World Heritage Expert Meeting on Visual Integrity in 2006

Contextual Facet	Value
Topic	Indian Architecture
Related Concepts / Objects	Shah Jehan, Mumtaz Mahal, Islam
Abstract Concept	love, death, devotion, remembrance
Context	built in memory of his favorite wife Mumtaz Mahal, by Shah Jehan; completed 1648

Temporal Facet	Value
Generic Time	summer, daytime
Specific Time	2006 (photo taken)

Figure 9.4: A faceted annotation of the Taj Mahal photo with generic, specific, and abstract values.

Some of these values can be read from the pixels: “outside”, “sky”, “daytime”, and “building” are visible to any careful observer, human or machine. Others, such as “UNESCO”, “Mumtaz Mahal”, or “completed 1648”, require an annotator who knows the context, because no analysis of the pixels can produce them. This is the second part of the semantic gap from The Semantic Gap, and it explains why metadata remains

valuable even when automated content analysis becomes very good. Facets also map directly to the filters and facet counts of Filtering, Facets, and Compression: a user narrows the results to "location: Agra" or "year: 2006" with one click.

Metadata Standards

Metadata is only useful across systems if everybody agrees where it is stored and what the fields mean. Several standards define the fields for the common media types:

Standard	Media	Typical fields	Written by
EXIF	Photos (JPEG, TIFF, HEIF)	Date and time taken, camera model, exposure, focal length, orientation, GPS coordinates	The camera, automatically
IPTC Photo Metadata (stored as XMP)	Photos	Headline, caption, keywords, creator, copyright, location, persons shown	Photographers, editors, agencies
Dublin Core	Any resource	15 elements: title, creator, subject, description, publisher, contributor, date, type, format, identifier, source, language, relation, coverage, rights	Libraries, archives, repositories
ID3	Audio (MP3)	Title, artist, album, year, genre, track number, cover image, lyrics	Labels, music software, users
schema.org	Web pages (JSON-LD)	Types such as `ImageObject`, `VideoObject`, `MusicRecording`: name, description, thumbnail, upload date, duration	Website owners, for search engines

EXIF is the metadata that every digital camera and smartphone writes without being asked. It is generic but precise: a timestamp and a GPS position answer the query "photos taken in Paris in 2023" without any content analysis. Many social platforms strip EXIF data on upload, mainly to protect the privacy of the uploader, so this valuable source often disappears on the way to the web. IPTC fields carry the editorial metadata of the news industry. They are usually stored inside the image file in XMP, an extensible metadata format that photo management software reads and writes. Dublin Core is the common denominator of libraries and digital archives. ID3 tags travel inside MP3 files and are the reason a music player shows title and artist. Finally, schema.org markup tells search engines what an embedded image or video shows; a `VideoObject` can even list the key moments of a video with their timestamps.

The standards fix the names of the fields, not their values. Two photographers who both fill in the IPTC keyword field may write "Taj Mahal" and "religious building". The standards solve the problem of where to find the description; they do not solve the problem of what the description says.

Curated Catalogues

In some domains, metadata is excellent. IMDb holds records for millions of movies and episodes. Each record follows a fixed schema with cast, crew, release dates, genres, plot summaries, and user ratings, and links to related records. Volunteers, actors, crews, and studios maintain the content. MusicBrainz is a community-maintained open encyclopedia of music with artists, releases, recordings, and their relation-

ships. Figure 9.5 shows its record for the song “Paparazzi” next to the lyrics of the song. Together, the two sources provide rich text features that cannot be derived from the video or audio signal alone.

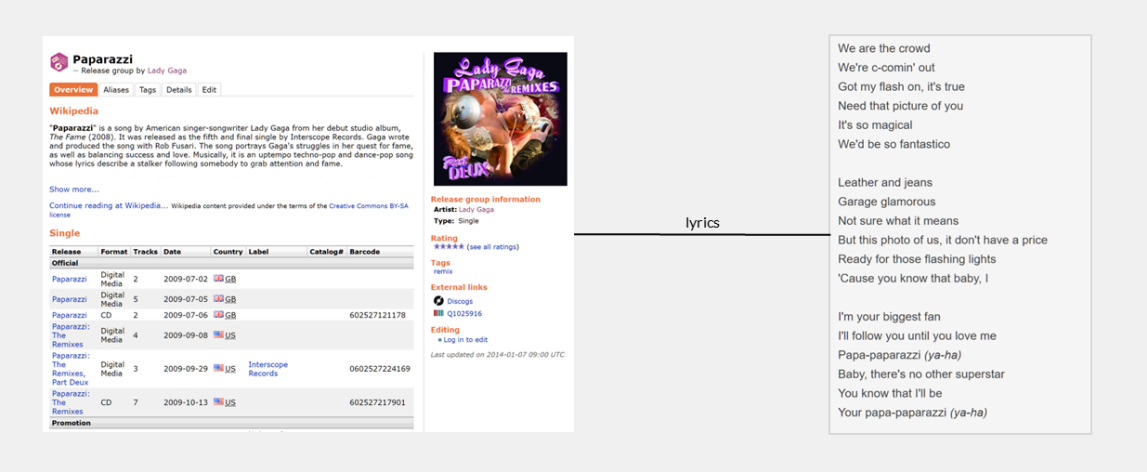


Figure 9.5: The MusicBrainz record for “Paparazzi” by Lady Gaga, linked to the lyrics of the song.

With curated catalogues, the metadata alone often bridges the semantic gap. Searching for a movie by its director or a song by its lyrics never touches the video or audio. The content itself is only needed for special tasks, such as identifying the song that is currently playing. Curated catalogues exist where commercial or community interest pays for the effort: the revenue of streaming platforms and the enthusiasm of fans cover the cost of the annotation. For an arbitrary photo collection, no one pays.

Manual Annotation Does Not Scale

Human annotation has become an industry because machine learning needs labeled training data, and these labels are metadata at the same time. Crowdsourcing platforms such as Amazon Mechanical Turk give access to over 500,000 independent workers who perform small, well-defined tasks for a fixed price. Figure 9.6 shows the setup of a labeling job in Amazon SageMaker Ground Truth. The requester chooses the task type (image classification, bounding boxes, semantic segmentation, or label verification), the workforce, and a price per task. The suggested price for a segmentation task that takes 90 seconds to 2 minutes is $0.84.

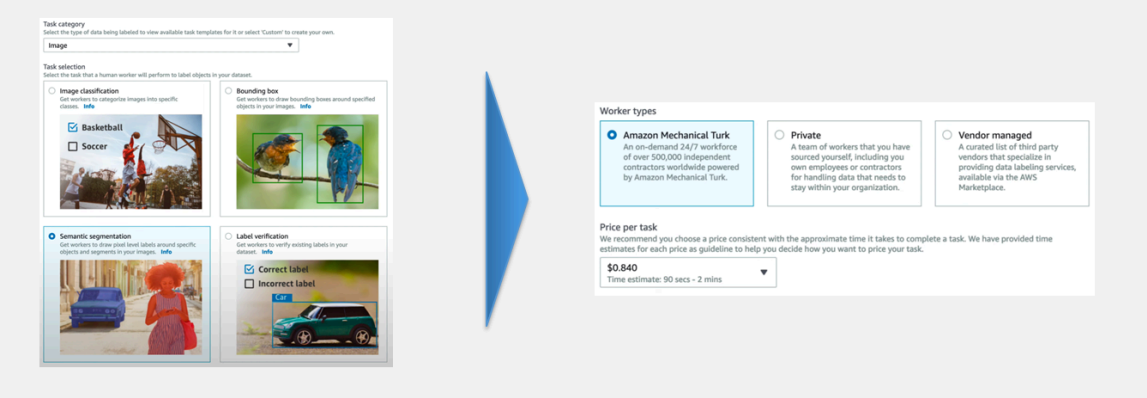


Figure 9.6: Configuring an image labeling job: task type, workforce, and price per task.

Example: The cost of annotating a photo archive

A media archive holds 1 million photos. At $0.84 per task, a single annotation pass costs $1,000,000 \cdot 0.84 = 840,000$ dollars. At 2 minutes per photo, it takes $1,000,000 \cdot 2/60 \approx 33,333$ hours of human work. Domain experts, such as archivists who recognize historical persons, cost far more than crowd workers. And the archive keeps growing every day.

Crowdsourcing scales the annotation to millions of items, but only for tasks that untrained workers can perform, and only for collections whose value justifies the cost. The fine-tuning of large language models

relies on the same workforce: thousands of workers rated the quality of generated answers. Once a model is trained on such labels, it produces labels automatically, which is the step that Automated Feature Extraction takes.

Cost is not the only problem; consistency is another. Suppose two workers annotate the Taj Mahal photo. Worker A writes "Taj Mahal, India, iconic building, 17th century". Worker B writes "religious building, great weather, few people outside, nice". Both annotations are correct, but they sit at different levels. Worker A knows the building and uses specific terms; Worker B describes what any observer sees and uses generic terms. Similar relationships appear everywhere in language: horse and mammal, Matterhorn and mountain, Italy and Europe. The embeddings of Building Semantic Search can relate "Taj Mahal" to "religious building", but only to a degree. Worker B's "nice" is an abstract and subjective judgment: two people rarely agree on it, yet it may be exactly what a user means with "great architecture in India".

The depth of the annotation varies as much as its level. Figure 9.7 contrasts a stock photo with a short list of keywords and an expert article that labels every architectural element of the building, from the finial on the dome to the calligraphy on the arches. The keyword list may be too thin to rank the photo for "Taj Mahal architecture"; the expert article is so detailed that most of its terms never appear in a query.

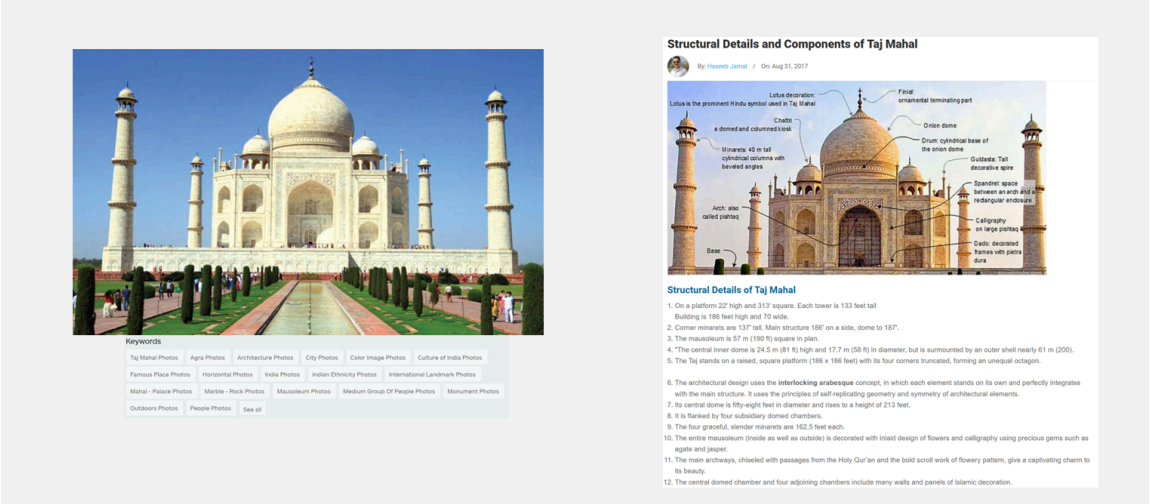


Figure 9.7: The same building annotated with stock photo keywords (left) and with an expert description (right).

Stock photo agencies and media archives therefore maintain concise keyword lists and add **faceted navigation** for the most requested facets: prominent persons, locations, brands, and decades. Annotators examine the photos of an event and tag only selected shots that show known persons. Figure 9.8 shows the benefit and the drawback. A journalist who needs a picture of the tennis player Darlene Hard for an obituary browses a curated folder instead of thousands of pictures. But a photo of children playing tennis today remains unannotated, because none of them is famous yet. If one of them becomes a champion in twenty years, the archive holds the early photos and cannot find them. The archivists of The Semantic Gap face the same limit: a clip that was not judged valuable when it was filed cannot be found when it is needed.

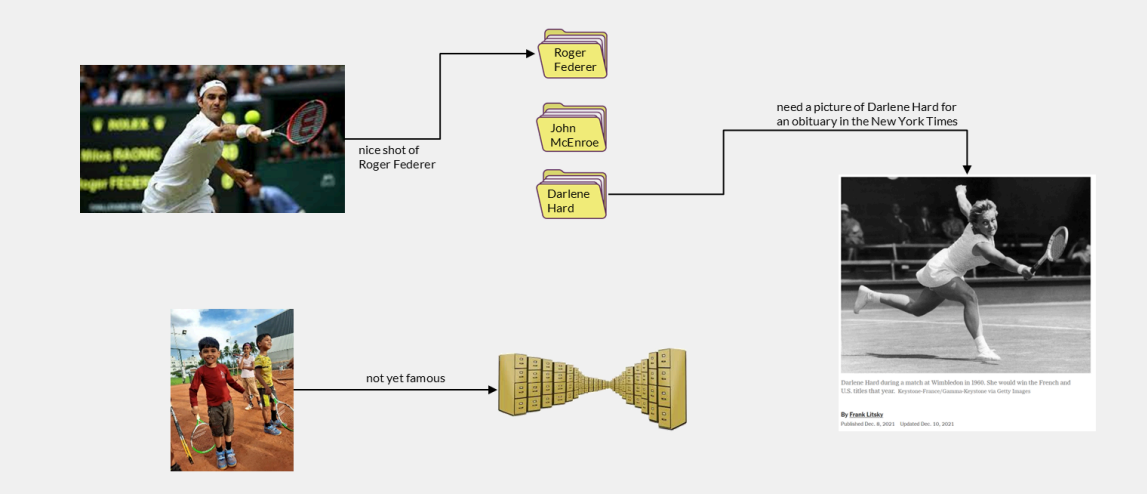


Figure 9.8: Faceted navigation by person finds annotated shots of known players, while photos of the not yet famous stay unfindable.

Harvesting Context from the Web

When images and videos are embedded in web pages, the page provides metadata at almost no cost. The HTML tags `<img>` and `<a>` carry attributes such as `alt` and `title` that describe the referenced object. Figure 9.9 shows a news article about the Taj Mahal next to its HTML source: the image has the title "Taj Mahal" and an `alt` text that repeats the headline, and a caption element below the image repeats the name. The headline, the article text, and the page title contain many more of the keywords a user would type.

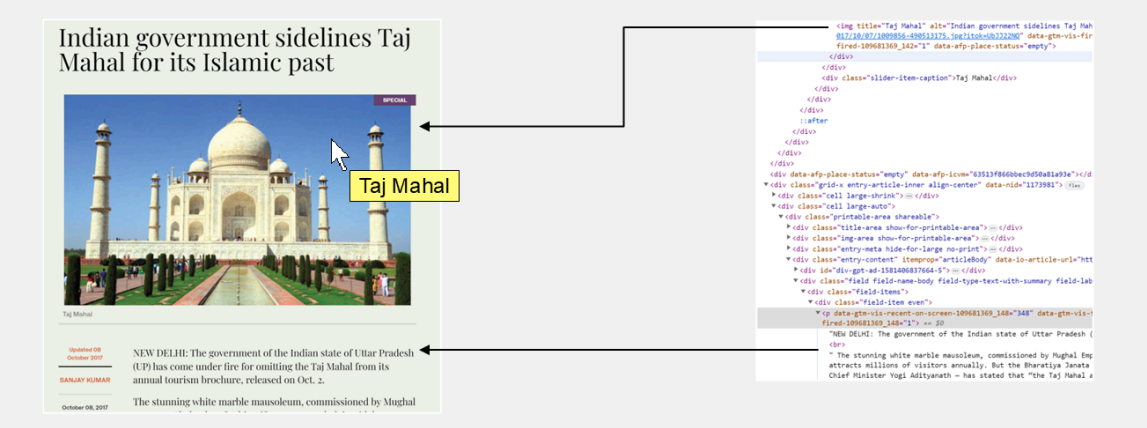


Figure 9.9: A news article and its HTML source: title, alt text, and caption describe the embedded image.

The difficulty is to decide which text belongs to the image. On early web pages, the describing text sat next to the `<img>` tag in the source, so a window of a few dozen tokens around the tag, plus the headers and the page title, captured most of the useful keywords. On modern pages, scripts and style sheets place elements on the screen independently of their position in the source. Extractors therefore judge proximity on the rendered page: text in the same visual zone as the image counts more than a sidebar with unrelated news, as Figure 9.10 shows. Web search engines add the anchor texts of links that point to an image, which describe it in the words of other authors, as Ranking Signals Beyond Term Counts explains.

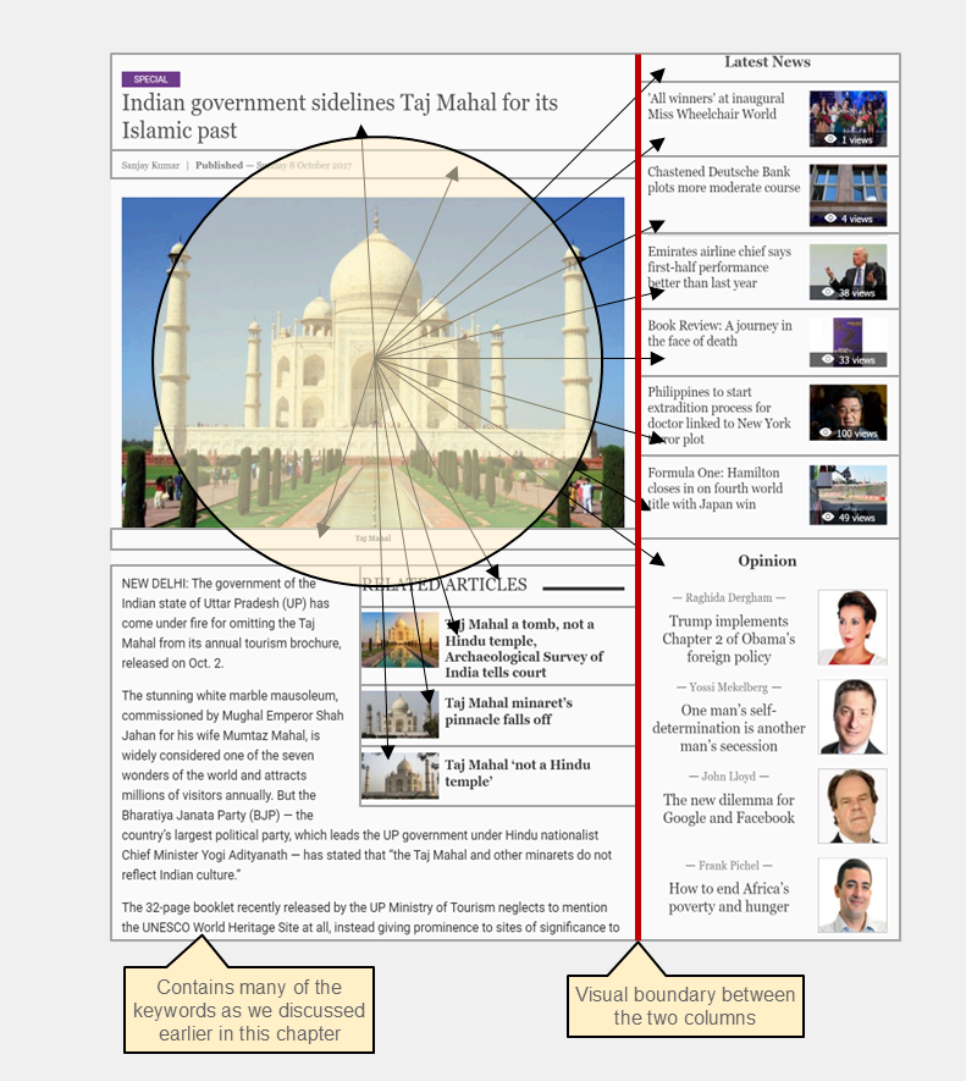


Figure 9.10: Visual zones of a news page: the article text near the image versus unrelated sidebars.

Harvested image-text pairs have since gained a second role. Open datasets such as LAION-5B collected billions of images together with their alt texts from web pages, and contrastive models such as CLIP in Multimodal Transformers learn from such pairs. The cheap and noisy metadata of the web became the training signal for the models that now match images and text without any metadata.

Can We Trust Metadata?

Harvested metadata is cheap, but it is written by people who may have their own goals. In 2001, Cory Doctorow listed in his essay "Metacrap" (see the Further Reading of the chapter summary) "seven insurmountable obstacles" between the world and a "meta-utopia" in which all content is accurately described:

1. **People lie.** Content creators publish misleading metadata to attract traffic.
2. **People are lazy.** Many publishers do not bother to annotate their content at all.
3. **People are stupid.** Many publishers lack the skill to catalogue their content well.
4. **Mission impossible: know thyself.** Even honest creators describe their own content inaccurately.
5. **Schemas aren't neutral.** Every classification scheme reflects the views and priorities of its designers.
6. **Metrics influence results.** Agreeing in advance on what to record and how to measure it decides what can be found later.
7. **There's more than one way to describe something.** Description is subjective; different perspectives are all valid.

This does not make metadata worthless. Curated catalogues such as IMDb and MusicBrainz are highly reliable, because their maintainers have an interest in quality and control the content. Observational

metadata, which describes an item in the words of others rather than its owner, is harder to manipulate. Web search engines trust the anchor texts of links from other pages more than the keywords the page owner chooses. Even this signal can be manipulated by coordinated campaigns, as the Google bombs described in Ranking Signals Beyond Term Counts showed. The lesson is to weigh each metadata source by how much it can be trusted, and to evaluate it before relying on it.

Metadata is a claim, not a fact

A keyword attached to an image states what somebody wanted the image to be found for. A common mistake is to treat metadata as ground truth when evaluating a retrieval system. Before using metadata as relevance labels or as filters, check who wrote it, what interest they had, and whether a second source agrees.

The keyword path thus fails in two ways. Manual annotation is too expensive for arbitrary collections, and harvested metadata cannot always be trusted. Both problems point in the same direction: describe the content from the content itself, automatically and consistently. The next section starts with the simplest form of such descriptions.

Hands-on: The semantic gap and metadata

See what a computer stores for an image, read the EXIF and IPTC/XMP metadata of a real photo, organize a faceted annotation, compare two annotators, harvest the text around an image on a web page, and catch a keyword-stuffed ad. At the end, check which queries each metadata source answers. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Automated Feature Extraction

Send the Taj Mahal photo to a commercial image analysis service, and within a second it returns two kinds of results. Figure 9.11 shows them. The perception branch lists the dominant colors with their shares (steel blue 35%, a light beige 22%, gray 14%) and measures brightness, sharpness, and contrast. The recognition branch lists labels with confidence values: “Arch” 96.5%, “Architecture” 96.5%, “Person” 94.9%, “Tomb” 78.5%, “Fun” 72.9%, “Monument” 65.7%, “Dome” 57.5%. For a second image, the service detects a face and estimates attributes such as age range and glasses; for a third, it recognizes the actress Brie Larson with 99% confidence.

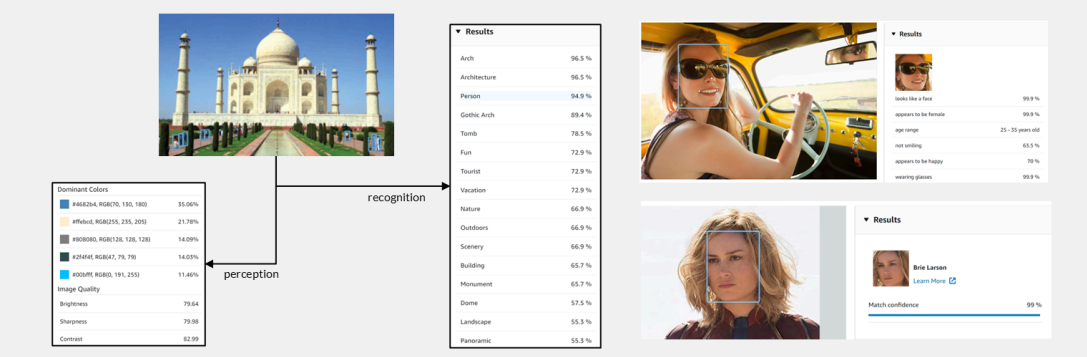


Figure 9.11: Automated analysis of images: perceptual features (dominant colors, quality) and recognition results (labels, face attributes, celebrity).

Nobody annotated these photos. The two branches correspond to two levels of automated content analysis. The colors describe how the image looks; the labels describe what it shows. This section walks up these levels, from features that describe perception to classifiers that produce labels, and ends with how neural networks took over the classification step.

The Abstraction Ladder

Automated content analysis works in layers. Each layer transforms the output of the layer below into a more abstract representation and closes a part of the semantic gap. Figure 9.12 shows the layers for a photo of a coyote on a snowy road.

Layer	Examples	Purpose
Raw-Level (Informational)	JPG, MP3, WAV, MP4, TXT	Store original content; basic digital representation
Low-Level (Perceptual)	Color histograms, edge maps, MFCCs, spectral centroid, textures	Capture sensory properties such as color, sound, or motion
Mid-Level (Structural)	Clustering, SVM, CNN/ViT embeddings, object or genre classification	Represent patterns, structure, and relationships within data
High-Level (Semantic)	Captioning, keyword generation, OCR, speech-to-text, topic extraction	Produce interpretable, concept-level meaning
Contextual (Metadata)	Timestamps, geolocation, device data, user tags, social context	Provide external descriptive context for indexing and filtering

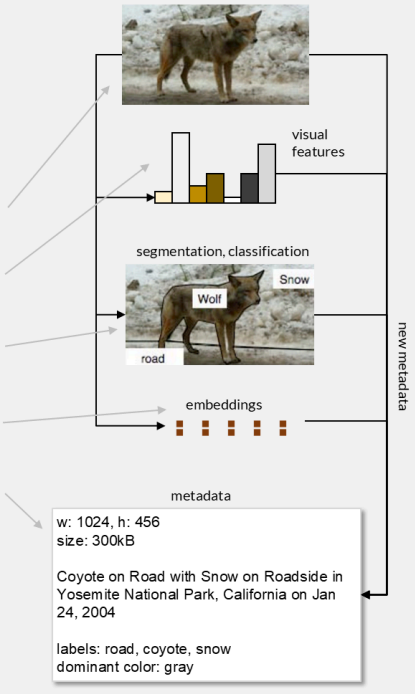


Figure 9.12: Five layers of content representation, from the raw file to semantic descriptions and metadata.

- **Raw level**: the file as stored, such as JPEG, MP3, or MP4. It holds pixel intensities or amplitude samples. This level preserves the content for playback but helps little with search, because similar content rarely means similar raw data: two photos of the same building, taken a few steps apart, share almost no identical pixel values.
- **Perceptual level** (low level): features that describe how the content looks or sounds, such as color histograms, textures, and edges for images, or the spectral properties of sound. They support comparisons such as "images with similar colors", but they do not interpret the content. Perceptual features invert the argument of the raw level: they assume that similar statistics of the signal indicate similar content. This assumption often holds, but not always.
- **Structural level** (mid level): patterns and categories recognized in the features, such as detected objects ("wolf", "road", "snow"), scene types, music genres, or shot boundaries in a video. Classifiers and learned embeddings work at this level.
- **Semantic level** (high level): interpretable descriptions such as captions, keywords, transcribed speech, or recognized text. At this level, the content is described in concepts that people use.
- **Contextual level**: information from outside the content, such as timestamps, locations, device data, user tags, and social context. Metadata from Metadata and Its Limits lives here, and the outputs of the higher levels are stored here as new metadata.

The layers show where each technique of this part of the book sits. The following subsections cover the perceptual and structural levels in general terms; the semantic level is the subject of Multimodal Transformers.

Perceptual Features

Perceptual features describe content the way humans perceive it. They do not attempt to recognize objects or interpret scenes. Instead, they measure properties of the signal that matter to human perception: the distribution of colors, the texture of surfaces, the shapes of edges, the pitch and timbre of a sound, the motion in a video. Each feature turns an item into a **feature vector**. A color histogram, for example, counts how many pixels fall into each of a fixed set of color bins, and the vector of counts describes the color impression of the image.

The idea is familiar from text retrieval. In Vector Space Retrieval, a document became a vector of term frequencies. A color histogram is the image counterpart of such a term vector: the bins play the role of terms, and the pixel counts play the role of term frequencies. The difference is that a term refers to a concept, while a color bin refers to a sensation. A red bin is high for a red car and for a sunset alike.

A system uses perceptual feature vectors in two ways:

1. **Similarity search.** The system compares feature vectors directly with a distance measure and returns the items closest to a query. This is **content-based retrieval**: the query is an example image, a sketch, a color palette, or a hummed melody instead of keywords. The distance measures and index structures of 6 - Vector Search apply directly. As Distance and Similarity Measures explains, the comparison function belongs to the feature: color histograms come with their own distances.
2. **Classification.** The system feeds the feature vectors into a classifier that predicts labels such as "outdoor", "speech", or "contains a face". The labels are text, so they return to the keyword path of Metadata and Its Limits, but now the system writes them automatically.

Good perceptual features must be stable under the variations that human perception ignores. A photo of a cat is still a photo of a cat after small changes, and a song is still the same song when played a little louder. The most important invariances are:

- **Translation**: an object shifted to a different position in the image is still the same object.
- **Rotation**: a slightly rotated coffee mug does not lose its identity.
- **Scale and resolution**: a tree in the background and the same tree close up, or a high-resolution photo and its thumbnail, show the same content.
- **Lighting and loudness**: a face in harsh sunlight and in soft indoor light is the same face; a song at two volume levels is the same song.
- **Noise and distortion**: sensor noise, compression artifacts, background noise in a recording, or a low-quality microphone should not change the feature much.

Each modality has its own perceptual features, and they are inspired by how human vision and hearing work. The following chapters develop them in detail: color, texture, and shape for images; time-domain, frequency-domain, and psychoacoustic features for audio; shot and motion features for video.

Perceptual similarity is not semantic similarity

A red car and a sunset have similar color histograms, while a black cat on a white carpet and a ginger cat on a green lawn have very different ones. It is tempting to expect perceptual features to find "similar content". They find content that looks or sounds similar, which is useful for queries by example, near-duplicate detection, or "more like this", but not for concept queries such as "pictures of cats".

Classification: From Features to Labels

Classification moves from the perceptual to the structural level. A classifier learns from labeled examples how feature values relate to classes and then predicts the class of new items. For two decades, the standard approach combined hand-crafted perceptual features with a classical learning algorithm such as a decision tree or a support vector machine.

A classic example is separating speech from music in a radio stream. Speech alternates between voiced sounds and short pauses, so its energy drops frequently; music keeps a more continuous energy level. The signal of speech also crosses zero at a more irregular rate than the signal of most music. A few features computed over short time frames, such as the share of low-energy frames and the variation of the zero-crossing rate, are enough for a decision tree to separate the two classes with high accuracy. The predicted labels "speech" and "music" become metadata, and they decide which analysis follows: speech goes to transcription, music to genre classification.

How well such a classifier works is measured with the confusion matrix and the metrics derived from it: accuracy, precision, recall, specificity, and the ROC curve. Evaluating Text Classifiers and Scores, Thresholds, and ROC Curves introduce them for text classifiers; they apply unchanged to image, audio, and video classifiers.

Automatically extracted labels are claims too. The classifier behind Figure 9.12 labeled the animal on the snowy road "wolf". It is a coyote, as the metadata at the bottom of the figure states. The two species look alike, and the classifier confused them. Better models reduce such errors but do not remove them. A classifier knows only the classes it was trained on and tends to confuse similar ones. Even the multimodal transformers of Multimodal Transformers, which can name almost anything, still mistake similar species, buildings, or persons, and sometimes describe details that are not in the image. Like the human metadata of Metadata and Its Limits, automatically extracted metadata must be evaluated before it is trusted.

A confidence value is not a probability of being right

The service in Figure 9.11 labels the Taj Mahal as "Gothic Arch" with 89.4% confidence, although the building is Mughal, not Gothic. For the face in the car, it reports "not smiling" with 63.5% and "appears to be happy" with 70%. Confidence values express how strongly the model favors a label, not how often such labels are correct. Before turning predicted labels into metadata, choose a threshold per label based on an evaluation against ground truth, as in Scores, Thresholds, and ROC Curves.

The weak point of this approach is the hand-crafted feature. A human expert must decide which properties of the signal separate the classes. That works for speech versus music, but not for the thousands of object classes that users search for. No one can design a feature that separates "Taj Mahal" from "other white buildings with domes". The breakthrough came when neural networks learned the features themselves.

Neural Networks: Key Steps

Neural networks for image classification went through several generations. Each one solved a specific obstacle of its predecessors, and most of them have since been replaced. Figure 9.13 shows the milestones, and the dropdown below tells the story in more detail.

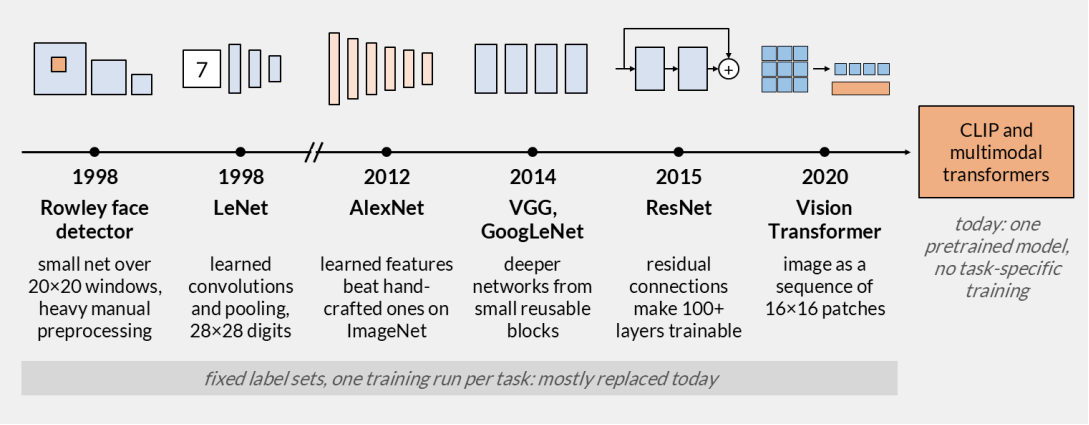


Figure 9.13: Milestones of neural image classification and what each step solved.

Several advances worked together. The ReLU activation function reduced the vanishing gradient problem that had blocked the training of deep networks. Large labeled datasets such as ImageNet and fast GPUs made it possible to train millions of parameters. Regularization methods such as dropout and data augmentation reduced overfitting, and better optimizers such as Adam made training faster and more stable. Convolutional networks exploit the spatial structure of images: they share weights across positions, which gives them translation invariance by design. Residual connections let the gradient flow back to early layers, so depth stopped being a problem.

The layers of a trained convolutional network form their own abstraction ladder. Early layers respond to edges and color blobs, much like the perceptual features above. Middle layers respond to textures and parts, and the last layers respond to whole objects. The output of the second-to-last layer is a dense vector that describes the image, and these vectors became the first learned image embeddings. Two photos of mountains have close vectors even when their color histograms differ.

From hand-wired face detectors to residual networks (optional reading)

Rowley, Baluja, and Kanade at Carnegie Mellon University built one of the first practical neural face detectors in 1998. To keep the network small, they trained it on preprocessed 20 by 20 pixel windows: each window was corrected for lighting, its histogram was equalized, and a second “router” network first estimated the rotation of a potential face so that the window could be rotated upright. At detection time, the system slid the window over the image at many positions and over a pyramid of downscaled copies to find faces of different sizes. The method worked well, but every step except the final decision was designed by hand.

LeNet, developed by Yann LeCun and colleagues for reading handwritten digits, showed that a network could learn the features itself. Its convolution layers apply small learned filters across the whole image, and its pooling layers reduce the resolution step by step. On the MNIST digits it reached accuracies close to human performance without any manual feature design. Yet for more than a decade, convolutional networks remained a niche: the images were small, the datasets were small, and training deeper networks failed because the gradients vanished on their way back to the early layers.

AlexNet changed this in 2012. It won the ImageNet Large Scale Visual Recognition Challenge, which asks for one of 1,000 classes per image, with a top-5 error of 15.3%, against 26.2% for the best entry based on hand-crafted features. It was deeper than LeNet, processed 224 by 224 pixel images, used the ReLU activation instead of saturating functions such as tanh, and was trained on two GPUs. Its two large fully connected layers needed most of its 60 million parameters. The result convinced the computer vision community within a year.

The following years were a race for depth. VGG from Oxford built networks from repeated blocks of 3 by 3 convolutions. GoogLeNet won the 2014 challenge with inception modules that run convolutions of several sizes in parallel and concatenate the results. ResNet added a shortcut around every block. If $f(\boldsymbol{x})$ is the mapping that a block should compute, the block learns only the residual $g(\boldsymbol{x}) = f(\boldsymbol{x}) -$

$\boldsymbol{x}$ and adds it to its input, so the output is $\boldsymbol{x} + g(\boldsymbol{x})$. If a block is not needed, it can learn $g = 0$, and the gradient flows back through the shortcut unchanged. ResNet trained networks with 152 layers and became the standard backbone for years. DenseNet went one step further and connected every layer to all following layers.

The Vision Transformer of 2020 finally removed the convolutions. It cuts an image into patches of 16 by 16 pixels, embeds each patch like a token, and processes the sequence with the same transformer encoder that Transformer Embeddings describes for text. With enough training data, it matches or beats convolutional networks, and it opened the door to models that process text and images with one architecture.

Most of these networks are no longer trained for retrieval systems today. Each one learned a fixed set of classes from a labeled dataset, and adding a new class meant collecting new labels and training again. Two developments replaced them. **Self-supervised learning** trains models on huge unlabeled or weakly labeled collections, for example by predicting masked parts of the input, so manual labels are no longer the bottleneck. **Contrastive learning** aligns different modalities in one embedding space, so that a model learns from image-caption pairs found on the web instead of from fixed label sets. The ideas of the older networks survive: convolutions, residual connections, and patches are building blocks of current models. The next section shows what these models do for retrieval.

Hands-on: Automated feature extraction

From pixels to labels: measure dominant colors, compare photos by raw pixels and by color histograms, test invariances, separate speech from music with a decision tree, and compare the features and labels of ResNet-50 with perceptual features. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Multimodal Transformers

Show a photo of the Matterhorn to a current multimodal language model and ask for a description. The answer reads like the work of a careful human annotator: “The image centers on a single, dramatic mountain peak with a sharp, pyramid-like summit. Its steep faces are streaked with snow and dark gray rock. The peak looks like the Matterhorn in the Swiss Alps, a shape recognized around the world.” The model recognized a specific named entity, described the scene, the colors, and the lighting, and did so without any task-specific training.

Transformers process sequences of tokens. Transformer Embeddings applied them to text; the Vision Transformer of Automated Feature Extraction applied them to sequences of image patches. Multimodal transformers combine both: they encode images, audio, or video together with text. For retrieval, they offer two paths across the semantic gap. The first path generates text that describes the content and then searches the text. The second path maps content and text into one shared embedding space and compares them directly. This section covers both and compares them.

Describe, Then Search Text

An **image-to-text model** has two parts. A visual encoder, usually a Vision Transformer, turns the image into a sequence of embeddings. A transformer decoder generates text token by token. Through cross-attention, each generated token looks at the image embeddings, so the words follow the visual content. Such models are trained on large collections of image-caption pairs. Current multimodal language models go one step further: the image embeddings enter a full language model together with a text prompt, so the user controls through the prompt what the model writes.

This control turns the model into a configurable metadata extractor. Figure 9.14 shows a two-step pipeline for the Matterhorn photo. The first prompt asks for a detailed description of the main subject, the setting, the colors, and the lighting. The second prompt asks for a JSON document with predefined fields: a headline, keywords, named entities, dominant colors and shapes, extracted text, and boolean flags such as `people_present` and `is_outdoors`.

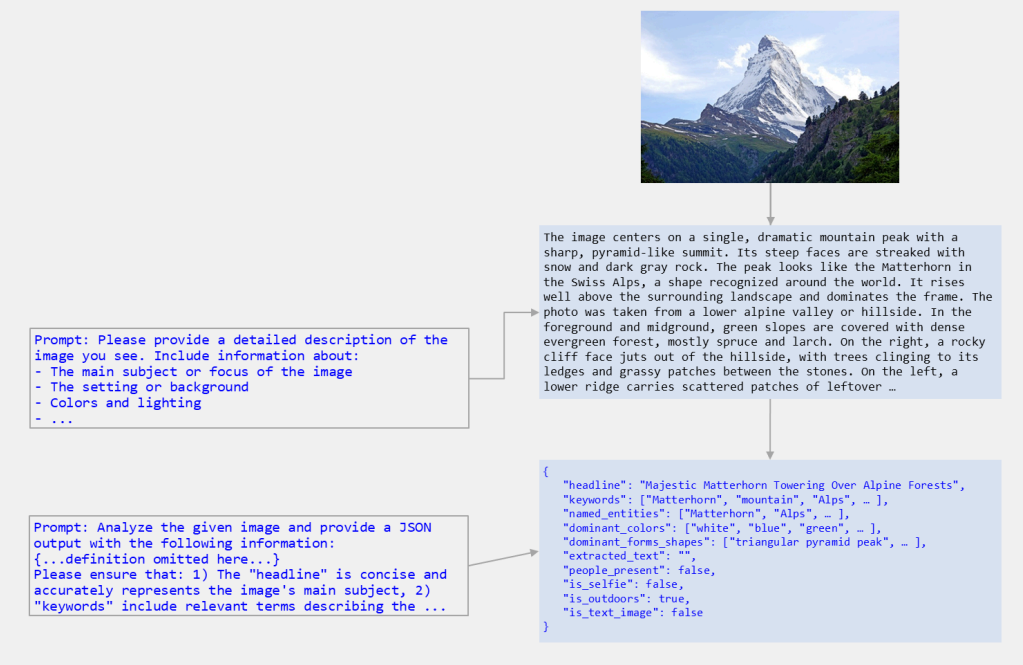


Figure 9.14: Two-step metadata extraction with a multimodal language model: a free-text description and a structured JSON document.

Each field of the JSON document has a place in the index of 4 - Index for Text Retrieval. The headline and the description become text fields for BM25 and for text embeddings. The keywords and named entities become fields for exact matching and facets. The boolean fields become filters: “outdoor photos without people” is a simple filter query. The prompt decides which metadata the archive gets, at which level, and in which format. Changing the schema means changing the prompt, not retraining a model. A good starting

point for writing such prompts is to ask the model itself to draft a prompt from a short specification, a technique called meta-prompting.

The same path works for other media. A speech recognition model such as Whisper transcribes the spoken words of a podcast, an interview, or a video into text with timestamps. It is a transformer that encodes the audio signal and decodes text tokens, trained on 680,000 hours of weakly labeled audio from the web. Text recognition (OCR) extracts the words shown in images and videos, from street signs to presentation slides. For video, a multimodal model can describe the frames of each shot. In all cases, the output is text, and the complete text toolkit applies: BM25, embeddings, filters, and the retrieval-augmented generation of 7 - Retrieval Augmented Generation.

The describe path has clear strengths. The metadata is readable, so an archivist can check and correct it. The search uses mature, cheap, and well-understood text indexes. Search results can be explained, because each hit matched specific words. But it also has a fundamental limit: the index contains only what the model chose to write at indexing time. If the description of a beach photo does not mention the red umbrella in the corner, the query "red umbrella" will not find the photo. Generating rich descriptions for every item is also expensive, because a large model must process each item once, and again whenever the prompt changes.

The Eighth Obstacle: Hallucination

Doctorow's seven obstacles in Metadata and Its Limits describe the weaknesses of human metadata, and Automated Feature Extraction showed that classifiers confuse similar classes. Generated metadata adds an eighth obstacle that is harder to spot: **models hallucinate**. A language model produces the most plausible continuation of its input, not a verified statement of fact. When the visual evidence is weak, it fills the gap with a plausible guess and states it with the same confidence as a correct observation. A mountain that resembles the Matterhorn may be labeled "Matterhorn"; a person in a suit at a lectern may be named after a well-known politician; text that is too small to read may be "extracted" anyway.

Hallucinations are especially harmful for specific metadata, such as names, places, and dates, because users trust exactly these fields. Several measures reduce the risk. The prompt can ask the model to mark uncertain values or to leave a field empty when the evidence is unclear. Generated values can be cross-checked against independent metadata: if the EXIF GPS coordinates place the photo in Canada, the label "Matterhorn" is wrong. Specific named entities can be restricted to a controlled list, such as the known persons of a news archive. For high-value collections, a human verifies the generated metadata, which is far cheaper than writing it from scratch.

Generated metadata looks better than it is

Generated descriptions are fluent, detailed, and consistent in style, so they look more reliable than the terse keywords of human annotators. It is tempting to judge the quality of generated metadata by reading a few examples. Evaluate it like any other classifier: sample items, compare the generated fields with ground truth, and measure precision and recall per field.

A Shared Embedding Space

The second path avoids the detour through text. **CLIP** (Contrastive Language-Image Pre-training), published by OpenAI in 2021, trains two encoders side by side: a text encoder (a transformer) that maps a caption to a vector, and an image encoder (a Vision Transformer or a convolutional network) that maps an image to a vector of the same dimension. The training goal is that an image and its caption end up close together in this shared space.

The training data are 400 million image-caption pairs collected from the web. In each training step, the model takes a batch of N pairs, encodes all images and all captions, and computes the cosine similarity of every image with every caption. This gives an $N \times N$ similarity matrix. The N entries on the diagonal belong to matching pairs; the other $N^2 - N$ entries belong to mismatched pairs. The contrastive loss rewards high similarity on the diagonal and low similarity everywhere else: matching embeddings are pulled together, mismatched ones are pushed apart. No one labels classes; the captions that people wrote

for their images on the web provide the supervision. Figure 9.15 shows a training batch as a similarity matrix and the zero-shot use with the Matterhorn example computed below.

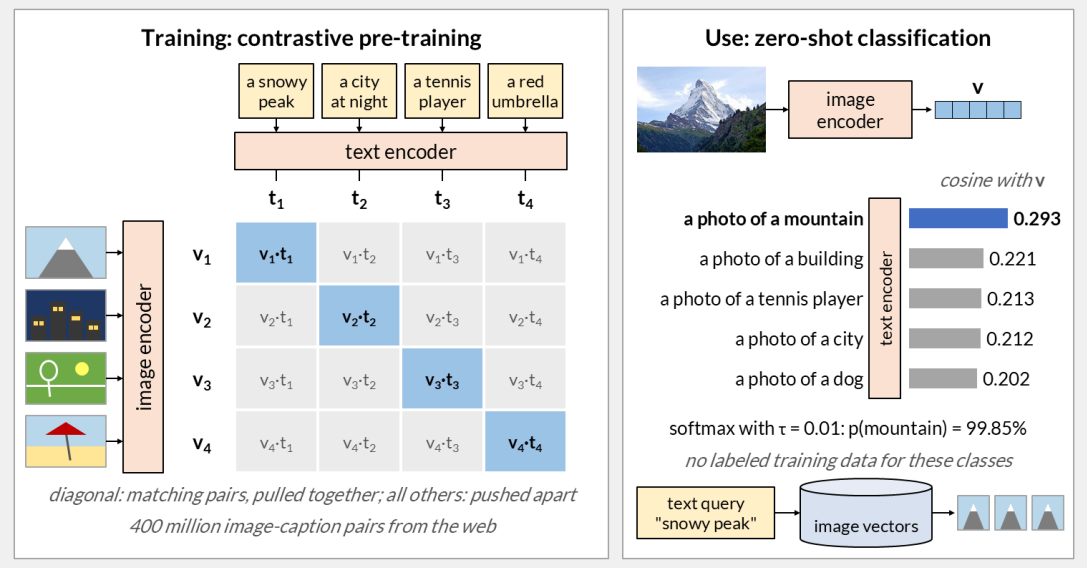


Figure 9.15: CLIP: contrastive training of an image encoder and a text encoder (left) and zero-shot classification with text prompts and text-to-image search (right).

What if two captions fit the same image?

The diagonal rule is a simplification. If a batch contains two photos of snowy mountains, the caption of the second photo counts as a mismatch for the first, although it describes it well. The loss tolerates such collisions. It is a softmax over each row, so it only asks that the matching caption scores highest: once it leads by a few hundredths of cosine similarity, the push on the other caption practically stops, and both captions stay close to both photos. Such collisions are also rare in batches of 32,768 random pairs, and over many steps every mountain photo pulls its own mountain caption closer.

After training, the shared space supports retrieval in every direction. For **text-to-image search**, the system encodes every image of the collection once and stores the vectors in a vector index from 6 - Vector Search. A text query such as “a snow-covered mountain at sunrise” is encoded with the text encoder, and a nearest-neighbor search returns the closest images. No keyword was ever attached to them. For **image-to-image search**, the query is an image, and the same index returns visually and semantically similar images. For **image-to-text search**, the system finds the captions or documents closest to an image.

The shared space also turns CLIP into a classifier for any set of classes, without training. For **zero-shot classification**, the system writes a short text prompt for each class, such as “a photo of a mountain”, encodes the prompts, and assigns the image to the class whose prompt is closest.

Key Formula: Zero-Shot Classification with CLIP

$$p(c \mid \text{image}) = \frac{e^{s_c}}{\sum_{c'} e^{s_{c'}}} = \frac{e^{s_c - s_{\max}}}{\sum_{c'} e^{s_{c'} - s_{\max}}} \quad \text{with} \quad s_c = \frac{\cos(\boldsymbol{v}, \boldsymbol{t}_c)}{\tau} \tag{9.1}$$

Encode the image as $\boldsymbol{v}$ and the prompt of each class c as $\boldsymbol{t}_c$, divide the cosine similarities by the temperature τ, and apply a softmax to the scores s_c. Subtracting the largest score $s_{\max}$ does not change the result, because the factor $e^{-s_{\max}}$ cancels in the numerator and the denominator, but it keeps every exponential between 0 and 1. The temperature is learned during training; for CLIP it is $\tau = 0.01$, so the scores are the similarities multiplied by 100.

Example: Classifying the Matterhorn photo

CLIP ViT-B/32 compares the Matterhorn photo with five prompts. The cosine similarities are 0.293 for "a photo of a mountain", 0.221 for "a photo of a building", 0.213 for "a photo of a tennis player", 0.212 for "a photo of a city", and 0.202 for "a photo of a dog".

Dividing by $\tau = 0.01$ gives the scores s_c = 29.3, 22.1, 21.3, 21.2, and 20.2. Subtracting $s_{\max} = 29.3$ gives the exponents 0, −7.2, −8.0, −8.1, and −9.1. Their exponentials are 1, 0.00075, 0.00034, 0.00030, and 0.00011, which sum to 1.0015. The class "mountain" receives $1/1.0015 = 99.85\%$, "building" 0.07%, and the other three classes 0.03% or less.

The same five prompts assign a photo of Roger Federer to "a photo of a tennis player" with 99.9998%: its similarity is 0.304, against 0.157 to 0.163 for the other prompts.

The absolute similarity values are low: even the best matches reach only 0.293 and 0.304. CLIP embeddings of images and texts occupy different regions of the shared space, so cross-modal cosine similarities typically fall between 0.1 and 0.4. What carries the information is the difference between the candidates, which the temperature amplifies. A threshold on the raw cosine, such as "relevant if above 0.5", would reject every image.

Contrastive training is not limited to images. CLAP (Contrastive Language-Audio Pre-training) applies the same idea to audio clips and their descriptions, so a query such as "dog barking in the rain" can retrieve sound recordings. Video models embed frames or short clips and align them with transcripts and captions. Newer models embed several modalities into one space.

Choosing a Path

The two paths answer different needs, and many systems use both.

Aspect	Describe, then search text	Shared embedding space
Index	Text index (inverted file), fields, filters	Vector index of media embeddings
Queries	Keywords, names, filters, boolean conditions	Free-text descriptions, example images
Coverage	Only what the description mentions	Any visual concept the model learned
Explainability	High: matched words are visible	Low: a similarity score only
Specific entities	Good, if the model names them correctly	Weak for rare names and exact identifiers
Indexing cost	High: a large generative model per item	Moderate: one encoder pass per item
Typical failure	Hallucinated or missing details	Plausible but wrong neighbors

The describe path excels at specific facts, filters, and explainable results: "photos of the Matterhorn taken in winter without people". The embedding path excels at visual concepts that nobody thought to describe: "a lonely bench in the fog". A hybrid system runs both and fuses the rankings with reciprocal rank fusion, as Building Semantic Search does for sparse and dense text retrieval. The generated descriptions can also serve as passages in a multimodal retrieval-augmented generation pipeline, where a language model answers questions about an image archive.

Hands-on: Describe or embed

Run the two-step metadata pipeline of Figure 9.14, turn the JSON fields into an index and evaluate them, provoke a hallucination, reproduce the CLIP zero-shot example of this section, search images by text, and send the same queries down both paths. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Time-Based Media and Specialized Solutions

A sports editor needs the goal from last night's match for the evening news. The archive holds the full recording: 90 minutes of play from several cameras, the stadium sound, and the live commentary. A search that returns "the match" is useless; the editor wants the 20 seconds around the goal. A music fan holds a phone towards the loudspeaker in a noisy bar and wants to know which song is playing. A search that returns "songs that sound similar" is useless too; the fan wants the one recording that is playing.

Both requests break assumptions of the previous sections. The first one adds time: the content unfolds over minutes or hours, and the user wants a moment, not a file. The second one asks for exact identification under heavy distortion, a task that general-purpose features and embeddings do not solve. This section covers both.

Time as a New Dimension

Images are static: the whole content is present at once. Audio and video unfold over time. They have a structure of transitions and segments, and a single file may contain hundreds of different moments. A user who searches a video collection usually wants a specific segment, just as a user of a document collection wants the relevant passage rather than a book of 500 pages. Time-based media therefore need the same step that Chunking Strategies introduced for long texts: split the content into retrieval units, index the units, and return the matching units with their position in the file.

Video has a natural hierarchy of such units:

- A **frame** is a single still image. At 25 to 60 frames per second, a video contains far too many frames to index them all individually, and neighboring frames are nearly identical.
- A **shot** is a continuous sequence of frames recorded by one camera without interruption. Shots are separated by cuts, where one frame switches abruptly to a different view, or by gradual transitions such as fades, dissolves, and wipes. The shot is the basic building block for indexing: a few representative keyframes per shot can be analyzed with the image techniques of the previous sections.
- A **scene** groups consecutive shots that share a location, a time, or a continuous action. A dialogue filmed with alternating views of the two speakers consists of many shots but forms one scene.
- An **episode** is a complete film or one part of a series, usually stored as a separate file with its own metadata.

Detecting shot boundaries is a perceptual task: the content of the frames changes abruptly or gradually. Detecting scenes is harder, because shots of the same scene can look very different. It requires semantic understanding of who is present, where, and what happens.

Audio needs units too, but it has no natural boundaries like cuts. Instead, the signal is split into overlapping **frames** of fixed length, for example 40 ms with a hop of 20 ms, and each frame yields a feature vector. Frames are then grouped into longer **segments**, for example 4 seconds with a hop of 100 ms, and statistics over the frame features describe each segment. A 4-second segment of 40 ms frames with a 20 ms hop contains $(4000 - 40)/20 + 1 = 199$ frames. The overlap plays the same role as the overlap of text chunks: an event that falls on the boundary of one unit lies in the middle of the next. Figure 9.16 shows both hierarchies next to the passages of a text document.

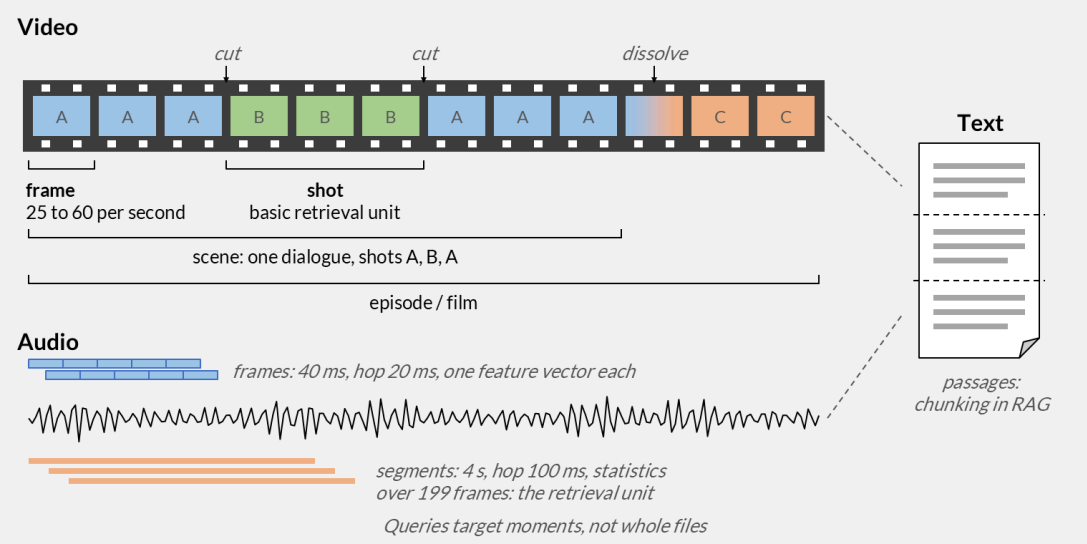


Figure 9.16: Retrieval units of time-based media: frames, shots, scenes, and episodes for video; frames and segments for audio.

Time also brings features that do not exist in a single image. **Spatio-temporal features** describe motion and activity: optical flow measures how pixels move between consecutive frames and reveals both object motion and camera motion. Three-dimensional convolutional networks extend convolutions from the image plane into time and learn features of short video volumes. Such features answer queries that no single frame can answer, such as clips of people running, car chases, or unusual movement in surveillance footage.

The modalities of a video are aligned on a common time line, and this alignment is a powerful retrieval tool. A transcript produced by speech recognition carries timestamps, so a text hit in the transcript leads directly to the moment in the video. The stadium sound and the commentary of the sports match locate the goal: the crowd noise rises sharply in energy, and the commentator speaks faster and at a higher pitch. A system that detects these audio events finds candidate highlights before it analyzes a single frame. Cross-modal transformers fuse frames, sound, and text over time and can answer queries such as “the moment when the reporter says X while standing in front of Y”. Very long videos remain a challenge: the sequences are long, the computational cost is high, and one noisy modality, such as unrelated background music, can mislead the fusion.

Specialized Solutions

The general approaches of this chapter describe content in a way that serves many queries. Some tasks have requirements that these descriptions cannot meet. Identifying a song from a phone recording is the classic example.

The query is a 10-second recording from a bar, with conversation, clinking glasses, and a phone microphone that compresses and distorts the sound. The catalogue holds millions of studio recordings. The answer must be exactly one recording: not a similar song, not another version by the same artist. Comparing waveforms fails because noise changes every sample. Comparing spectral envelopes fails for the same reason. An embedding of musical similarity, in the spirit of Multimodal Transformers, would happily return other songs of the same genre and mood. The task needs features that ignore heavy distortion and are at the same time specific enough to distinguish millions of recordings, and it needs a lookup in the full catalogue that answers within a second.

Audio fingerprinting, the technique behind Shazam, meets these requirements. It extracts a compact fingerprint from the parts of the signal that survive noise and compression, precomputes the fingerprints of all catalogued recordings, and looks up the fingerprint of a query in a hash index. Figure 9.17 (left) names the stages of the pipeline; the chapter on audio features explains how each stage works and why it is robust.

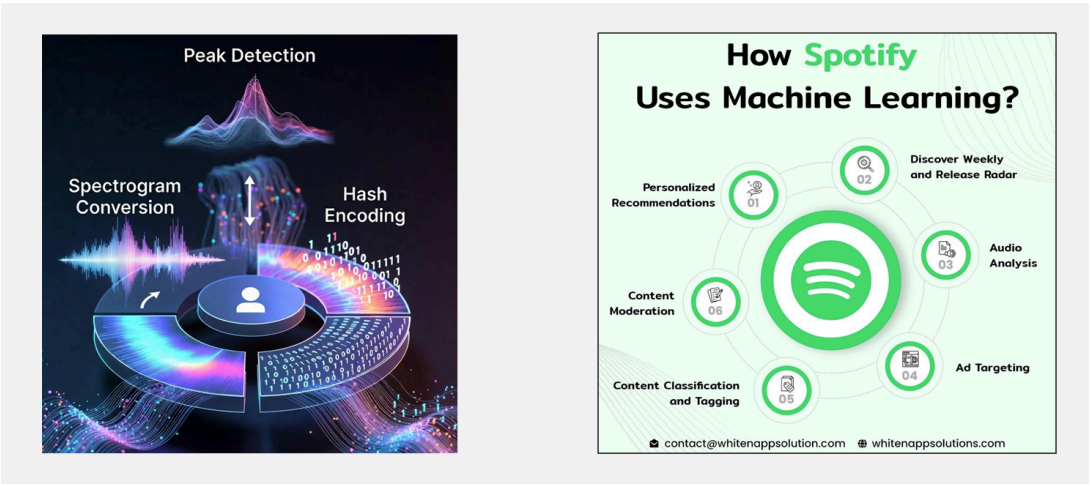


Figure 9.17: The stages of audio fingerprinting (left) and the machine learning applications of a music streaming service (right).

The specialization has a price. Fingerprinting matches recordings, not songs. When users sing a song themselves, their pitch, timing, and timbre differ from the recording, and the fingerprint no longer matches. Live performances with different instruments, tempo, and audience noise often fail for the same reason. Query by humming needs a different solution that compares melodies rather than recordings. A specialized solution solves its task very well and little else.

Music streaming services show the opposite strategy: they combine many signals. Figure 9.17 (right) lists the machine learning applications of Spotify, from personalized recommendations to content classification. To recommend a track, the service combines acoustic features such as tempo, key, timbre, and energy, higher-level attributes such as genre, mood, and instrumentation, text such as artist information, user tags, and lyrics, and the listening behavior of millions of users. A listener may like a song for the timbre of the voice, the rhythm, the lyrics, or its cultural associations, so the system must model several kinds of similarity at once.

Other specialized solutions follow the same pattern. Face recognition identifies persons across an archive with dedicated face embeddings. Perceptual hashing finds near-duplicate images, such as resized or recompressed copies of a photo. Copyright detection on video platforms matches uploads against fingerprints of protected content. In each case, a precisely defined task with strict requirements on robustness, exactness, or speed justifies a dedicated method.

When to look for a specialized solution

General features and embeddings answer "what is similar to this?" A specialized method is worth considering when the task asks "which exact item is this?", when the query is heavily distorted, or when a very large catalogue must answer within a second.

Summary

Approach Comparison

The chapter followed one question: how does a retrieval system built for text find images, audio, and video? Each approach bridges the semantic gap at a different level and with different costs.

Approach	Bridges the gap with	Scales to arbitrary collections	Trust	Best for
Manual annotation	Human-written keywords and facets	No: about $1 per item	High for curated catalogues	Archives, stock photos, IMDb, MusicBrainz
Harvested context	Captions, alt texts, surrounding text, anchor texts	Yes, for embedded media	Variable: owners can lie	Web images and videos
Perceptual features	Feature vectors of colors, textures, sounds	Yes	Objective, but no meaning	Query by example, near duplicates
Classical classifiers	Labels from hand-crafted features	Yes, per trained class	Measured with a confusion matrix	Narrow tasks such as speech versus music
Describe, then search text	Generated captions, JSON fields, transcripts	Yes, at high indexing cost	Fluent, but hallucinations	Names, facts, filters, explainable results
Shared embeddings (CLIP)	Text and media in one vector space	Yes	No readable evidence	Free-text visual queries, zero-shot labels
Specialized solutions	Task-specific features and indexes	Yes, for one task	High within the task	Exact identification (fingerprinting, faces)

The approaches build on each other rather than compete. Metadata provides context that no content analysis can recover. Perceptual features describe how content looks and sounds and feed classifiers. Classifiers and generative models write metadata automatically. Shared embeddings cover the visual concepts that nobody described. Time-based media apply all of them per segment, and specialized solutions take over where a task demands exactness or robustness beyond general methods.

Key Takeaways

1. The semantic gap is the distance between the features a system computes from a signal and the concepts people use in queries. Text carries its meaning in its symbols; images, audio, and video do not, so their meaning must be extracted before it can be matched. Part of the gap, such as "UNESCO World Heritage Site", cannot be closed from the content at all and needs context: metadata, the surrounding web page, or the world knowledge of a large model.
2. Metadata turns a multimedia search into a text search. Values are generic, specific, or abstract and are organized in facets. Standards such as EXIF, IPTC, Dublin Core, ID3, and schema.org define where metadata is stored, not what it says. Curated catalogues such as IMDb and MusicBrainz show how well the keyword path works when someone pays for quality.
3. The keyword path breaks for arbitrary collections. Manual annotation costs about a dollar per item and varies in level and depth between annotators; harvested metadata from the web is cheap but can be wrong or manipulated, as Doctorow's seven obstacles describe.
4. Automated content analysis climbs an abstraction ladder from raw data to perceptual, structural, semantic, and contextual representations. Perceptual feature vectors support similarity search

directly or serve as input to classifiers that produce labels. They must be invariant to changes that human perception ignores, and perceptual similarity is not semantic similarity.

5. Neural networks replaced hand-crafted features with learned ones, from LeNet and AlexNet to ResNet and the Vision Transformer. Most of these task-specific classifiers are superseded by models trained with self-supervised and contrastive learning, whose ideas they share. Their labels are claims too: classifiers confuse similar classes, such as a coyote labeled "wolf", and must be evaluated like human metadata.
6. Multimodal transformers offer two paths. The describe path generates captions, structured JSON fields, or transcripts and reuses the text toolkit, with readable but possibly hallucinated metadata. The embedding path maps media and text into one space (CLIP), enabling text-to-image search and zero-shot classification without labels. Hybrid systems combine both.
7. Audio and video need retrieval units: frames, shots, scenes, and episodes for video, overlapping frames and segments for audio, in analogy to text chunking. Some tasks, such as identifying a song from a noisy phone recording, need specialized solutions like audio fingerprinting that solve one task very well and little else.

Key Formulas

Key Formula: Zero-Shot Classification with CLIP

$$p(c \mid \text{image}) = \frac{e^{s_c}}{\sum_{c'} e^{s_{c'}}} \quad \text{with} \quad s_c = \frac{\cos(\boldsymbol{v}, \boldsymbol{t}_c)}{\tau} \tag{9.2}$$

Compare the image embedding with the embedding of a text prompt per class; the softmax with temperature $\tau = 0.01$ turns small cosine differences into a clear decision.

Exam focus

- The two parts of the semantic gap: concepts visible in the content versus context that the content does not contain, and why the gap differs between text and other media.
- Generic, specific, and abstract metadata; facets; what metadata standards such as EXIF and IPTC define and what they leave open.
- Why manual annotation does not scale, why annotations by different workers disagree, and why harvested metadata cannot be fully trusted (Doctorow's obstacles; classifier confusions; hallucination as the eighth obstacle).
- The levels of the abstraction ladder, and the two uses of perceptual features: similarity search and input to classification.
- The describe path versus the shared embedding path of multimodal transformers: index type, queries, coverage, explainability, and typical failures.
- How CLIP is trained with a contrastive loss over image-caption pairs, and how zero-shot classification works.
- Why time-based media need retrieval units, and when a specialized solution is preferable to general features.

Self-Check Questions

1. (Understand) For each query in Figure 9.1, decide whether a perfect analysis of the pixels could answer it, or whether it needs context from outside the image. Explain your decision.
2. (Understand) Classify the following metadata values for a photo of the Matterhorn as generic, specific, or abstract: "mountain", "Zermatt", "majestic", "1865 first ascent", "snow", "winter".
3. (Apply) An archive of 250,000 photos is annotated by crowd workers at \$0.84 per photo, and every photo is annotated by three workers to resolve disagreements. Compute the cost and the human hours if each task takes 2 minutes.
4. (Apply) CLIP returns cosine similarities of 0.25, 0.24, and 0.20 between an image and three class prompts. Compute the zero-shot probabilities with $\tau = 0.01$. What changes if $\tau = 1$?
5. (Analyze) A user searches a news photo archive for "the minister shaking hands at the 2019 summit". Which parts of this query are best served by EXIF data, by generated JSON metadata, and by CLIP embeddings?

6. (Analyze) Why does an audio fingerprint identify a studio recording in a noisy bar, but fail when the user sings the song? What would a system need to compare instead?
7. (Evaluate) A startup proposes to replace all human metadata in a broadcast archive with descriptions generated by a multimodal language model. Assess the proposal: what does the archive gain, what does it risk, and how would you evaluate the generated metadata before deciding?

Test Your Knowledge

Take the Chapter 9 Quiz ->

Further Reading

- Smeulders, A. W. M., Worring, M., Santini, S., Gupta, A., & Jain, R. (2000). **Content-based image retrieval at the end of the early years**. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(12), 1349-1380. IEEE Xplore. The survey that defined the semantic gap and framed content-based image retrieval for the following decade.
- Doctorow, C. (2001). **Metacrap: Putting the torch to seven straw-men of the meta-utopia**. Online essay. The original list of the seven obstacles to trustworthy metadata in Metadata and Its Limits.
- Schuhmann, C., et al. (2022). **LAION-5B: An open large-scale dataset for training next generation image-text models**. *Advances in Neural Information Processing Systems*, 35, Datasets and Benchmarks Track. arXiv. 5.85 billion image and alt-text pairs harvested from web pages, the kind of data on which the CLIP models of Multimodal Transformers are trained.
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). **ImageNet classification with deep convolutional neural networks**. *Advances in Neural Information Processing Systems*, 25. NeurIPS proceedings. AlexNet: the result that made learned features replace hand-crafted ones.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). **Deep residual learning for image recognition**. *IEEE Conference on Computer Vision and Pattern Recognition*, 770-778. arXiv. Introduces the residual connection that made very deep networks trainable.
- Dosovitskiy, A., et al. (2021). **An image is worth 16x16 words: Transformers for image recognition at scale**. *International Conference on Learning Representations*. arXiv. The Vision Transformer, which treats image patches as tokens and underlies the image encoders of current multimodal models.
- Radford, A., et al. (2021). **Learning transferable visual models from natural language supervision**. *International Conference on Machine Learning*, 8748-8763. arXiv. The CLIP paper: contrastive training on 400 million image-text pairs and zero-shot classification with text prompts.
- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). **Robust speech recognition via large-scale weak supervision**. *International Conference on Machine Learning*, 28492-28518. arXiv. Whisper, the transformer that turns speech into searchable text for the describe path.
- Wang, A. (2003). **An industrial-strength audio search algorithm**. *International Conference on Music Information Retrieval*. PDF. The Shazam fingerprinting algorithm, the specialized solution of Time-Based Media and Specialized Solutions.
- IPTC. **IPTC Photo Metadata Standard**. iptc.org. The field definitions behind the editorial metadata of news photos.
- DCMI. **DCMI Metadata Terms**. dublincore.org. The Dublin Core elements and their refinements.
- Schema.org. **VideoObject**. schema.org. The markup that tells search engines what a video on a web page shows, including its key moments.